

DEVOPS MASTER COURSE

Volume 11D

OpenTelemetry, Distributed Tracing, APM, Profiling and eBPF

*Context Propagation - OTLP - Collector Engineering - Sampling - Tempo - TraceQL -
Pyroscope - Hubble - Full-Stack Diagnosis*

Chapters 31-40 | 2026 Edition

Course Standard Instrument responsibly, preserve context, control cardinality and cost, validate every telemetry hop, rehearse failure, and prove that traces and profiles shorten diagnosis.

Beginner to Production-Ready DevOps and SRE Engineering

About This Part

Volume 11D adds request-level causality and code-level performance evidence to the metrics and logging platform built in Parts A-C. It begins with trace structure and context propagation, standardizes instrumentation through OpenTelemetry, engineers resilient Collector pipelines, controls sampling and privacy, operates Grafana Tempo, and then integrates application performance monitoring, continuous profiling, and eBPF network visibility.

The goal is not maximum telemetry volume. The goal is sufficient, trustworthy evidence to answer: which request failed, where time was spent, what dependency was involved, which code path consumed resources, and whether the network or policy layer contributed to the incident.

2026 Architecture Note Tempo 3.0 distributed deployments use a Kafka-compatible message queue between ingestion and downstream trace-processing components. Treat queue sizing, durability, authentication, lag, and recovery as part of the tracing platform design.

Volume 11 Roadmap

Part	Primary Scope	Status
11A	Observability foundations, Linux monitoring, telemetry, SLIs and SLOs	Completed
11B	Prometheus, PromQL, exporters, rules and Alertmanager	Completed
11C	Grafana, Loki, Elastic/OpenSearch and centralized logging	Completed
11D	OpenTelemetry, tracing, APM, profiling and eBPF	This book
11E	SRE operations, incidents, capacity, reliability and governance	Next
11F	Enterprise projects, certifications, interviews and capstone	Planned

Scope, Safety, Privacy and Cost Controls

- Use synthetic users, test services, disposable namespaces, and non-production telemetry backends for laboratories.
- Never place secrets, authorization headers, session identifiers, payment data, health data, or unnecessary personal identifiers in span attributes, baggage, profile labels, or network-flow exports.
- Apply attribute allowlists, redaction, hashing, or deletion before telemetry leaves the trust boundary. Baggage is propagated in network headers and must be treated as externally visible context.
- Define trace and profile ingest budgets, sampling targets, retention, tenant limits, query limits, object-storage lifecycle, and emergency throttling procedures.
- Use stable semantic conventions, bounded attributes, and controlled resource metadata. Unbounded user IDs, URLs, SQL text, stack labels, or pod UIDs can create cost and privacy risks.
- Protect OTLP receivers, Tempo endpoints, Pyroscope endpoints, Kafka-compatible queues, object storage, Grafana data sources, Hubble Relay, and debug interfaces with authentication, encryption, network policy, and least privilege.
- Profile and eBPF agents require elevated host access in many designs. Review kernel compatibility, capabilities, host mounts, security policy, and organizational approval before deployment.
- Pin collector distributions, instrumentation agents, operators, backend versions, and configuration schemas; test upgrades and rollback in staging.

Part D Learning Outcomes

- Explain traces, spans, links, events, status, resources, attributes, context propagation, baggage, exemplars, and asynchronous causality.
- Standardize service identity and telemetry names using OpenTelemetry resources and semantic conventions.
- Choose manual, library, zero-code, operator-injected, or eBPF instrumentation based on coverage, risk, and maintenance requirements.

- Design agent, gateway, and agent-to-gateway Collector topologies with memory limiting, batching, retry, queues, persistent storage, health checks, and internal telemetry.
- Implement head and tail sampling policies that preserve errors and high-value transactions while controlling cost and bias.
- Operate Tempo with appropriate deployment mode, object storage, queue architecture, multi-tenancy, retention, compaction, query performance, backups, and security controls.
- Query traces with TraceQL and correlate traces with metrics, logs, service graphs, span metrics, exemplars, deployments, and runbooks.
- Use continuous profiling and flame graphs to identify CPU, allocation, memory, lock, and code-path inefficiencies.
- Use eBPF-based profiling and Hubble network observability while understanding privilege, portability, symbolization, and coverage limitations.
- Build and validate an integrated full-stack observability capstone with controlled failure scenarios and recovery evidence.

Table of Contents

1. Chapter 31: Distributed Tracing Fundamentals and Context Propagation
2. Chapter 32: OpenTelemetry Architecture, Resources and Semantic Conventions
3. Chapter 33: Instrumentation Engineering: Manual, Automatic and Kubernetes Operator
4. Chapter 34: OpenTelemetry Collector Pipelines, Topologies and Resiliency
5. Chapter 35: Sampling, Baggage, Privacy and Telemetry Cost Control
6. Chapter 36: Grafana Tempo Architecture, Storage, Security and Operations
7. Chapter 37: TraceQL, Exemplars, Service Graphs and Cross-Signal Correlation
8. Chapter 38: Application Performance Monitoring and Production Diagnosis
9. Chapter 39: Continuous Profiling with Grafana Pyroscope
10. Chapter 40: eBPF Observability and Full-Stack Telemetry Capstone

VOLUME 11D | CHAPTER 31

Chapter 31: Distributed Tracing Fundamentals and Context Propagation

Reconstructing request causality across services, queues, databases and asynchronous work

Trace Data Model

A distributed trace is a tree or graph-like record of work triggered by a transaction. Spans represent timed operations. Parent-child relationships describe nested work; links represent causal relationships that are not naturally hierarchical, such as batch consumers or fan-in processing. A span normally contains a trace ID, span ID, parent span ID, name, kind, start and end time, status, attributes, events, links, and resource identity.

Element	Purpose	Design Rule
Trace ID	Correlates all spans for one distributed transaction	Propagate across every supported boundary; never regenerate mid-request.
Span name	Describes an operation	Use stable low-cardinality names, not raw URLs or user values.
Attributes	Searchable context	Allowlist bounded business and technical dimensions.
Events	Timestamped facts inside a span	Record exceptions, retries, cache misses, or state transitions.
Links	Causal connection without parent-child hierarchy	Use for queues, batches, retries, and asynchronous joins.
Resource	Identity of the entity producing telemetry	Set service.name, environment, version and deployment identity consistently.

Context Propagation

Context propagation carries trace identity across process boundaries. HTTP, gRPC, messaging, scheduled work, and background jobs need explicit injection and extraction through supported propagators. The default interoperable choice is W3C Trace Context. Mixed environments may temporarily require B3 or vendor formats during migration, but multiple propagators increase ambiguity and testing burden.

Trace context and log correlation example

```
# Example request headers
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
tracestate: vendorname=value

# Application logs should copy trace identifiers from active context
{"level": "ERROR", "trace_id": "4bf92f...", "span_id": "00f067...", "message": "payment timeout"}
```

Synchronous and Asynchronous Boundaries

- HTTP and gRPC usually preserve a parent-child chain when client and server instrumentation agree on propagation.
- Message producers should inject context into message metadata. Consumers may create a child span for one message or use links when processing batches or delayed work.
- Retries should be represented as events or child spans so latency and repeated dependency load remain visible.
- Fan-out creates multiple child branches. Fan-in should preserve links to all relevant upstream contexts rather than inventing a false single parent.
- Trace continuity is not proof of correctness. Validate timestamps, clocks, service identity, span kinds, errors, and missing branches.

Production Readiness Checklist

- Document supported protocols and propagators.
- Set stable service and operation names.
- Test context across every synchronous and asynchronous boundary.
- Correlate traces with structured logs without leaking sensitive values.
- Measure broken-trace rate, orphan spans, clock skew and export failures.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Trace stops at one service	Missing extraction or unsupported library	Compare inbound headers and active span context	Enable compatible instrumentation or inject/extract manually.
Two traces for one request	Context regenerated or propagator mismatch	Compare trace IDs at client and server	Standardize propagators and startup configuration.
Huge number of operations	Raw paths or IDs used in span names	Group span names by frequency	Use route templates and bounded operation names.
Negative or impossible duration	Clock skew or timestamp misuse	Compare host time and span timestamps	Synchronize clocks and use SDK-managed timing.

Hands-On Lab

1. Deploy three small services with one HTTP call and one queue hop.
2. Instrument the entry service and verify one trace ID across all services.
3. Break propagation deliberately at the queue and document the symptom.
4. Repair propagation, add an error event, and correlate one log line.
5. Export a trace diagram and write the request timeline.

Validation Evidence

- Trace containing HTTP and messaging spans.
- Headers or message metadata with sensitive values removed.
- Before-and-after evidence for the broken propagation drill.
- Sanitized structured log containing trace_id and span_id.

Interview Questions and Model Answers

Q: Why are links needed?

A: They represent causality that is not accurately modeled by one parent, especially batches, queues, retries and fan-in.

Q: What makes a good span name?

A: A stable operation name based on a route or logical action, with variable data stored only in controlled attributes.

Chapter Summary

- Tracing reconstructs causality, not merely latency.
- Propagation must be tested at every boundary.
- Stable names and resource identity are essential for useful queries.

VOLUME 11D | CHAPTER 32

Chapter 32: OpenTelemetry Architecture, Resources and Semantic Conventions

Building a vendor-neutral telemetry contract across languages and platforms

Core Components

OpenTelemetry separates telemetry creation from storage and analysis. APIs are used by application code and instrumentation libraries. SDKs implement sampling, processors and export. Instrumentation libraries observe frameworks and dependencies. OTLP transports telemetry. The Collector receives, transforms and exports signals. Backends store and query them. This separation allows platform teams to change backends without rewriting every application.

Layer	Responsibility	Common Failure
API	Creates spans, metrics, logs and context operations	Application depends directly on a backend-specific API.
SDK	Sampling, processors, resource detection and export	Multiple SDK initializations or conflicting exporters.
Instrumentation library	Observes framework or dependency behavior	Duplicate spans from overlapping instrumentations.
OTLP	Vendor-neutral telemetry transport	TLS, endpoint, protocol or compression mismatch.
Collector	Receive, process, route and export telemetry	Configured components are not enabled in service pipelines.
Backend	Store, search, aggregate and visualize	Retention, tenant, index or query design is unsuitable.

Resources and Service Identity

Resources describe the entity producing telemetry. The minimum reliable contract is a stable service.name plus environment, service version, deployment or instance identity, and platform attributes that support ownership and rollout comparison. Resource detection should enrich rather than overwrite deliberate application identity.

Environment-based resource and exporter configuration

```
OTEL_SERVICE_NAME=checkout-api
OTEL_RESOURCE_ATTRIBUTES=deployment.environment.name=production,service.version=2026.08.1,service.namespace=commerce
OTEL_EXPORTER_OTLP_ENDPOINT=https://otel-gateway.example.internal:4317
OTEL_EXPORTER_OTLP_PROTOCOL=grpc
```

Semantic Conventions

Semantic conventions standardize attribute names and operation meaning for HTTP, databases, messaging, RPC, cloud, containers, Kubernetes and other domains. They enable reusable dashboards and queries, but conventions

evolve. Pin instrumentation versions, record convention stability, test migrations, and avoid creating organization-specific aliases for fields already covered by the standard.

Contract Area	Required Governance
Service identity	Naming registry, owner, tier, environment and version rules.
HTTP and RPC	Route templates, method, status and peer attributes without sensitive headers.
Database	System and operation identity; redact statements and bind values by default.
Messaging	System, destination template, operation and message handling semantics.
Errors	Consistent status and exception recording; do not mark expected outcomes as failures.
Schema evolution	Version-aware rollout, compatibility tests and dashboard migration plan.

Production Readiness Checklist

- Define an organization resource contract.
- Adopt official semantic conventions before custom fields.
- Pin SDK and instrumentation versions.
- Test duplicate instrumentation and resource precedence.
- Publish a telemetry schema change process.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
unknown_service appears	service.name absent or overwritten	Inspect resource attributes at backend	Set OTEL_SERVICE_NAME and verify detector precedence.
Dashboards split one service	Inconsistent namespace, case or environment names	Group by resource attributes	Normalize identity at deployment templates or Collector.
Duplicate spans	Two agents or manual plus auto instrumentation overlap	Compare instrumentation.scope values	Disable overlapping library or remove duplicate manual spans.
Queries break after upgrade	Semantic convention change	Diff attributes before and after upgrade	Pin versions and migrate dashboards with compatibility tests.

Hands-On Lab

1. Create a telemetry contract for five services.
2. Instrument two languages using the same resource attributes.
3. Query by service, environment and version.
4. Upgrade one instrumentation package in a test namespace and compare attributes.
5. Write a schema migration note and rollback plan.

Validation Evidence

- Resource attribute inventory.
- Before-and-after semantic convention comparison.
- Query proving consistent service identity.
- Approved telemetry contract document.

Interview Questions and Model Answers

Q: Why are semantic conventions valuable?

A: They make telemetry portable and reusable by standardizing names and meanings across languages and tools.

Q: What is the difference between resource and span attributes?

A: Resource attributes identify the producing entity; span attributes describe one operation or request.

Chapter Summary

- OpenTelemetry is an interoperability contract, not a storage backend.
- Resource identity is foundational for every signal.
- Semantic conventions require lifecycle governance.

VOLUME 11D | CHAPTER 33

Chapter 33: Instrumentation Engineering: Manual, Automatic and Kubernetes Operator

Selecting the least risky method that still captures useful application behavior

Instrumentation Strategies

Method	Strength	Limitation	Best Fit
Manual	Precise business operations and custom attributes	Requires code changes and ownership	Critical workflows and domain events.
Library instrumentation	Reusable framework and dependency coverage	Version compatibility must be managed	Shared HTTP, DB, messaging and RPC libraries.
Zero-code agent	Fast coverage with minimal source change	May miss business context or create duplicates	Brownfield services and rapid baseline adoption.
Kubernetes Operator injection	Centralized rollout of auto-instrumentation	Webhook, image and runtime compatibility risks	Governed cluster environments.
eBPF auto-instrumentation	Low-touch visibility for supported Linux workloads	Privilege, language and semantic limitations	Discovery and brownfield coverage, not full business tracing.

Manual Instrumentation Pattern

Python manual span example

```
from opentelemetry import trace
tracer = trace.get_tracer("commerce.checkout", "1.0.0")

with tracer.start_as_current_span("checkout.validate_cart") as span:
    span.set_attribute("commerce.cart.item_count", len(items))
    span.set_attribute("commerce.checkout.channel", channel)
    validate(items)
    span.add_event("cart_validated")
```

Manual spans should describe business operations that automatic instrumentation cannot infer. Attributes must be bounded and reviewed. Avoid customer names, email addresses, raw order IDs, full URLs, SQL statements, request bodies, or tokens. Prefer a controlled business category, result, tier, or feature flag.

Kubernetes Operator Pattern

Illustrative Operator-managed instrumentation resource

```
apiVersion: opentelemetry.io/v1alpha1
kind: Instrumentation
metadata:
  name: platform-default
  namespace: commerce
spec:
  exporter:
```

```

    endpoint: http://otel-agent:4318
    propagators: [tracecontext, baggage]
    sampler:
      type: parentbased_traceidratio
      argument: "0.10"
  ---
  # Workload annotation example
  instrumentation.opentelemetry.io/inject-python: "platform-default

```

Operator injection changes workload startup. Validate init containers, injected libraries, runtime versions, musl versus glibc requirements, startup time, memory overhead, side effects, and rollback. Use canary namespaces and admission-webhook monitoring before broad rollout.

Instrumentation Acceptance Tests

- One inbound server span and one outbound client span per real call.
- Correct service, route, method, status, dependency and environment identity.
- Context survives supported HTTP, RPC and messaging boundaries.
- No sensitive headers, request bodies or database values are exported.
- Overhead remains inside an agreed CPU, memory and latency budget.
- Application behavior is unchanged when exporter or Collector is unavailable.

Production Readiness Checklist

- Create a per-language instrumentation support matrix.
- Canary auto-instrumentation before fleet rollout.
- Test duplicate spans and missing context.
- Define manual-span review standards.
- Measure overhead and startup impact.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Pod fails after injection	Runtime or injected image incompatibility	Inspect init container and application startup logs	Use supported image, runtime and libc combination or disable injection.
No spans despite injection	Annotation, Instrumentation resource or endpoint wrong	Describe workload and operator resources	Correct reference, namespace and OTLP endpoint.
Application latency rises	Excess spans, synchronous export or heavy attributes	Compare profiles and span rate	Reduce instrumentation, batch export and tune sampling.
Database data leaked	Statement capture enabled without redaction	Inspect exported attributes	Disable or sanitize statement and bind-value capture.

Hands-On Lab

1. Deploy a Python or Java sample service without tracing.
2. Enable zero-code instrumentation in a test namespace.
3. Add one manual business span and controlled attributes.
4. Measure baseline and instrumented latency and memory.
5. Break the exporter endpoint and verify the application remains healthy.

Validation Evidence

- Workload manifest and Instrumentation resource.
- Trace showing automatic and manual spans.
- Overhead comparison table.
- Failure test proving exporter isolation.

Interview Questions and Model Answers

Q: When should manual instrumentation be used?

A: For domain operations and business context that framework-level automatic instrumentation cannot understand.

Q: What is the main rollout risk of auto-instrumentation?

A: It changes application startup and runtime behavior, so compatibility, duplication, overhead and rollback must be tested.

Chapter Summary

- Use the simplest instrumentation method that meets diagnostic needs.
- Auto-instrumentation accelerates coverage but does not replace business context.
- Instrumentation must fail safely when telemetry infrastructure is unavailable.

VOLUME 11D | CHAPTER 34

Chapter 34: OpenTelemetry Collector Pipelines, Topologies and Resiliency

Operating telemetry collection as a reliable production data plane

Collector Component Model

Collector configuration defines receivers, processors, exporters, connectors and extensions. A component is inactive until referenced by a service pipeline or extension list. Processor order is operationally significant: memory limiting should occur early, privacy transforms before export, and batching near the end. Validate configuration with the Collector validate command before rollout.

Collector trace pipeline example

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 768
    spike_limit_mib: 192
  attributes/privacy:
    actions:
      - key: http.request.header.authorization
        action: delete
      - key: user.email
        action: delete
  batch:
    send_batch_size: 2048
    timeout: 5s

exporters:
  otlp/tempo:
    endpoint: tempo-gateway.observability.svc:4317
    tls:
      insecure: true
    sending_queue:
      enabled: true
    retry_on_failure:
      enabled: true

extensions:
  health_check:
    endpoint: 0.0.0.0:13133

service:
  extensions: [health_check]
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, attributes/privacy, batch]
      exporters: [otlp/tempo]
```

Deployment Topologies

Pattern	Advantages	Risks	Typical Use
Direct SDK to backend	Simple beginning	Backend coupling and weak policy control	Small experiments only.
Node or host agent	Local buffering, host metadata, low network distance	Per-node capacity and upgrade management	Kubernetes DaemonSet or host service.
Central gateway	Central policy, routing and backend credentials	Network dependency and shared bottleneck	Regional or cluster aggregation.
Agent to gateway	Local collection plus centralized control	More components and duplicate queue decisions	Common production standard.
Dedicated stateful sampling tier	Consistent tail decisions	Requires trace affinity and careful scaling	High-volume tail sampling.

Resiliency and Backpressure

Sending queues and retry reduce loss during transient backend failure. Persistent file-backed queues can survive Collector restart when loss is unacceptable, but require disk capacity, lifecycle monitoring and recovery tests. Queues are not infinite storage. Define maximum outage tolerance, drop policy, alert thresholds and emergency sampling or filtering actions.

- Monitor accepted, refused, dropped and failed telemetry by signal.
- Alert on exporter queue utilization, enqueue failures, retry exhaustion, memory limiter refusals, process memory, CPU, restarts and configuration reload failures.
- Tail sampling and other stateful processors cannot be scaled like stateless gateways; route all spans for a trace to the same decision context.
- Protect OTLP endpoints with TLS, authentication, network policy and bounded request size. Do not expose debug exporters or zPages publicly.
- Load test with realistic span size, burst shape, compression, tenant mix and backend outage duration.

Production Readiness Checklist

- Use `memory_limiter` and `batch` in production pipelines.
- Enable queue and retry with explicit capacity assumptions.
- Monitor Collector internal telemetry.
- Separate stateless routing from stateful tail sampling where scale requires it.
- Validate config, security and rollback before deployment.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Collector restarts under load	Memory limiter absent or queue oversized	Inspect memory and internal telemetry	Set realistic memory limits, queue size and batch parameters.
Spans dropped during outage	Queue full or retry duration exceeded	Check exporter enqueue and send-failure metrics	Increase justified capacity or reduce ingest with emergency policy.
Configured processor has no effect	Not referenced in pipeline	Inspect <code>service.pipelines</code> config	Add processor in correct order and validate config.
Tail sampling inconsistent	Spans for one trace reach different samplers	Compare routing and trace IDs	Use trace-aware load balancing or dedicated stateful tier.

Hands-On Lab

1. Run an agent and gateway Collector.
2. Send synthetic traces through both tiers.
3. Simulate backend outage and observe queue growth and retry.

4. Restart the gateway with and without persistent queue.
5. Load test and document the safe sustained and burst rate.

Validation Evidence

- Validated Collector configuration.
- Internal telemetry dashboard.
- Queue outage and restart test results.
- Capacity model with drop thresholds.

Interview Questions and Model Answers

Q: Why place `memory_limiter` early?

A: It applies backpressure before later processing increases memory pressure and helps prevent process termination.

Q: Why is `tail sampling` stateful?

A: The processor must retain spans until enough of the trace is available to make one trace-level decision.

Chapter Summary

- Collectors are production services and need SLOs, capacity and runbooks.
- Queues delay loss but do not remove capacity limits.
- Stateful processors require topology-aware scaling.

VOLUME 11D | CHAPTER 35

Chapter 35: Sampling, Baggage, Privacy and Telemetry Cost Control

Preserving diagnostic value without collecting every request or propagating unsafe context

Sampling Models

Model	Decision Point	Strength	Risk
Always on	SDK start	Complete traces	High cost and backend pressure.
Trace ID ratio	SDK start	Predictable probabilistic rate	Rare errors may be missed.
Parent based	SDK start using parent decision	Consistent distributed decision	Bad upstream decisions propagate.
Rule-based head	SDK start using known request data	Keeps selected routes or tenants	Cannot know final latency or result.
Tail sampling	After spans arrive at Collector	Can preserve errors, slow traces and important attributes	Memory, latency, affinity and state complexity.

Tail Sampling Policy

Illustrative tail-sampling policy

```
processors:
  tail_sampling:
    decision_wait: 15s
    num_traces: 50000
    policies:
      - name: keep-errors
        type: status_code
        status_code:
          status_codes: [ERROR]
      - name: keep-slow
        type: latency
        latency:
          threshold_ms: 1500
      - name: baseline
        type: probabilistic
```

```
probabilistic:
  sampling_percentage: 5
```

Tail sampling introduces decision delay and memory pressure. The decision wait must accommodate normal trace completion without holding data indefinitely. Capacity must be modeled using incoming trace rate, spans per trace, average span size, decision window and burst factor. Preserve a random baseline so healthy traffic remains comparable.

Baggage and Privacy

Baggage is propagated alongside context and may appear in network headers. It is not automatically a safe attribute store. Use only bounded, non-secret values needed across services. Never propagate access tokens, email addresses, customer names, raw account identifiers, authorization decisions, or regulated data. Prefer an internal opaque category or lookup key when cross-service context is necessary.

Control	Implementation
Allowlist	Only approved attribute and baggage keys may leave the service.
Redaction	Delete or transform headers, query values, DB statements and exception text before export.
Sampling budget	Assign baseline and exception sampling targets per service tier.
Tenant limits	Apply ingest, search, retention and concurrency limits per tenant.
Audit	Sample telemetry payloads in a protected test environment and record policy violations.
Emergency mode	Pre-approved config to reduce sampling or drop low-value spans during overload.

Production Readiness Checklist

- Define sampling objectives per service tier.
- Preserve errors and a healthy random baseline.
- Model tail-sampler memory and decision delay.
- Create an attribute and baggage allowlist.
- Test emergency cost-control configuration.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Errors absent from traces	Head sampling dropped them before outcome known	Compare error logs and trace coverage	Increase rate or use tail policy for eligible traffic.
Tail sampler OOM	Decision window or num_traces undersized for load	Inspect memory and pending traces	Scale stateful tier, reduce wait, rate or span volume.
Sensitive value appears in headers	Unsafe baggage key	Capture sanitized request headers in test	Remove baggage use and deploy allowlist processor.
Trace costs spike	New high-volume service or excess spans	Break down spans by service and operation	Apply budgets, reduce duplicate spans and tune sampling.

Hands-On Lab

1. Generate normal, slow and error requests.
2. Apply head sampling and measure which traces remain.
3. Apply tail sampling and verify error and slow-trace retention.
4. Inject an unsafe baggage key in a lab, detect it, then block it.
5. Create a capacity and cost worksheet for the sampling tier.

Validation Evidence

- Sampling comparison results.
- Trace set proving policy behavior.
- Privacy test and corrected allowlist.
- Tail-sampler capacity calculation.

Interview Questions and Model Answers

Q: Why keep a random baseline?

A: It supports comparison of normal behavior and prevents the dataset from containing only failures.

Q: Why is baggage risky?

A: It propagates across service boundaries in headers and can expose data beyond the original process or trust zone.

Chapter Summary

- Sampling is a product and reliability policy, not only a cost switch.
- Tail sampling preserves outcome-aware traces but adds state.
- Privacy controls must act before export and propagation.

VOLUME 11D | CHAPTER 36

Chapter 36: Grafana Tempo Architecture, Storage, Security and Operations

Operating a scalable trace backend with object storage, queueing and tenant controls

Tempo Deployment Modes

Tempo can run as a monolithic process for laboratories and small deployments or as distributed components for production scale. In current Tempo 3.0 distributed architecture, distributors write incoming traces to a Kafka-compatible queue, while downstream components consume, build blocks and serve live data. Object storage holds trace blocks. The queue and object store are therefore first-class reliability dependencies.

Component or Dependency	Responsibility	Operational Focus
Distributor	Accepts and validates incoming traces	Availability, tenant auth, rate limits and queue write failures.
Kafka-compatible queue	Buffers ingestion and decouples downstream work	Durability, partitions, lag, authentication and recovery.
Block builder / ingester path	Builds durable trace blocks	Backlog, flush failures, memory and object-store writes.
Query frontend / querier	Plans and executes searches	Concurrency, timeouts, cache and expensive queries.
Compactor	Compacts blocks and enforces retention	Backlog, object-store permissions and delete safety.
Object storage	Durable trace storage	Encryption, lifecycle, replication, cost and recovery objectives.

Baseline Configuration Concepts

Tempo lab configuration - verify version-specific schema before production

```
# Illustrative single-binary lab configuration
server:
  http_listen_port: 3200

distributor:
  receivers:
    otlp:
      protocols:
```

```

    grpc:
    http:

storage:
  trace:
    backend: local
    local:
      path: /var/tempo/traces

compactor:
  compaction:
    block_retention: 72h

```

Tempo does not provide an authentication layer by itself. Production deployments require an authenticating reverse proxy or gateway, encrypted transport, tenant identity controls and backend authorization. Separate read and write credentials where possible. Protect object storage and queue credentials with workload identity or a managed secret system.

Operations and Capacity

- Choose monolithic mode only when scale and availability requirements permit; distributed mode adds dependencies and operational complexity.
- Track ingest bytes, spans, rejected traces, queue write failures, consumer lag, block build failures, object-store errors, compaction backlog, query latency and query cancellations.
- Set tenant ingest limits, burst limits, max trace size, search limits and retention according to service tier.
- Test object-store outage, queue outage, component restart, credential rotation, retention deletion and restoration of configuration.
- Back up configuration, tenant policy and encryption material. Object storage replication is not a substitute for tested recovery procedures.

Production Readiness Checklist

- Select deployment mode from measured scale and availability needs.
- Design Kafka-compatible queue and object storage as critical dependencies.
- Place authenticated gateway in front of Tempo.
- Set tenant limits and retention.
- Monitor ingest, lag, blocks, compaction and query path.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Ingest succeeds but traces not searchable	Consumer lag or block/live-store issue	Check queue lag and downstream component health	Restore consumers and verify block build or live query path.
401/tenant errors	Gateway or tenant header mismatch	Inspect sanitized proxy logs and headers	Correct identity mapping and tenant configuration.
Queries timeout	Wide search, insufficient query capacity or storage latency	Inspect query frontend and object-store metrics	Narrow queries, add dedicated columns/capacity and tune limits.
Retention not deleting	Compactor backlog or permissions	Inspect compactor logs and storage access	Repair permissions and safely drain backlog.

Hands-On Lab

1. Deploy Tempo in monolithic lab mode.
2. Send traces through the Collector and query them in Grafana.
3. Place a reverse proxy in front and test unauthorized access.
4. Simulate object-storage or local-volume unavailability.
5. Create a distributed production design including queue, storage and recovery objectives.

Validation Evidence

- Tempo configuration and architecture diagram.
- Unauthorized-access test.
- Backend outage timeline.
- Capacity and recovery design for queue and object storage.

Interview Questions and Model Answers

Q: Why does Tempo need an external authentication layer?

A: Tempo itself does not include one, so production access must be protected by a trusted proxy or gateway.

Q: What changes operationally with Tempo 3.0 distributed mode?

A: A Kafka-compatible queue becomes a core ingestion dependency requiring durability, security, capacity and lag monitoring.

Chapter Summary

- Tempo is simple to start but distributed operation depends on queue and object storage reliability.
- Authentication and tenant control are external design responsibilities.
- Query and retention behavior require explicit limits and monitoring.

VOLUME 11D | CHAPTER 37

Chapter 37: TraceQL, Exemplars, Service Graphs and Cross-Signal Correlation

Moving from a symptom to the exact trace, log event, dependency and deployment change

TraceQL Investigation Patterns

TraceQL selects traces and spans by intrinsic fields, resource attributes, span attributes, duration, status and structural relationships. Start with narrow service, environment and time boundaries. Then filter by error, route, dependency, version or duration. Broad regular-expression queries over long periods are expensive and should be constrained by platform limits.

TraceQL investigation examples

```
# Error traces for checkout in production
{ resource.service.name = "checkout-api" && status = error }

# Slow server spans
{ kind = server && duration > 1s }

# Database spans inside traces that also contain a server error
{ span.db.system = "postgresql" } && { kind = server && status = error }

# Count matching spans over time (feature availability depends on Tempo version)
{ resource.service.name = "checkout-api" } | count_over_time()
```

Metrics from Traces

Span metrics produce request, error and duration measurements from spans. Service graphs derive relationships and RED signals between services. These capabilities are valuable when application metrics are missing, but generated metrics inherit the quality and sampling bias of trace data. Do not silently replace authoritative application metrics without documenting the differences.

Correlation Path	Purpose	Required Field or Configuration
Metric exemplar -> trace	Open a representative trace from a latency or error spike	Exemplar with trace ID and Tempo data source link.
Trace -> logs	Inspect events for the same service and time window	trace_id in logs plus configured derived field or query.
Trace -> metrics	View service or resource behavior around a span	Service identity and configured metric query mapping.
Trace -> profile	Find code paths active during a selected span	Profile integration, matching service labels and supported runtime.
Deployment annotation -> trace	Compare before and after a release	Version resource attribute and deployment event timeline.
Service graph -> trace search	Move from dependency edge to request evidence	Consistent client/server spans and peer identity.

Grafana Correlation Design

- Use one canonical service.name across Prometheus, Loki, Tempo and Pyroscope labels.
- Keep environment and version dimensions compatible across signals.
- Configure trace-to-logs and trace-to-metrics links using explicit fields, not fragile free-text assumptions.
- Validate both directions: metric to trace, trace to logs, trace to metrics and trace to profile.
- Record query cost and latency for common incident paths. Precompute span metrics or service graphs where justified.

Production Readiness Checklist

- Create standard TraceQL runbook queries.
- Configure exemplars and trace correlations.
- Validate service graphs against known architecture.
- Document sampling bias in generated metrics.
- Set query limits and expensive-query guidance.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Trace-to-logs returns nothing	Logs lack trace_id or label mapping is wrong	Search raw logs for trace ID	Fix log correlation and data-source mapping.
Service graph missing edge	Client/server spans or peer attributes absent	Inspect trace span kinds and attributes	Correct instrumentation and semantic conventions.
TraceQL query slow	Wide time range or expensive expression	Inspect query metrics and narrow conditions	Use bounded attributes, dedicated columns or stricter limits.
Exemplar link fails	Trace ID format or data-source mapping wrong	Inspect exemplar labels	Align exemplar field and Tempo data-source configuration.

Hands-On Lab

1. Create a latency spike with one slow dependency.
2. Open the trace from a metric exemplar.
3. Navigate from trace to the correlated log.
4. Generate service graph and span metrics.
5. Write a five-minute diagnosis runbook using the correlation path.

Validation Evidence

- Screenshot or export of exemplar-to-trace navigation.
- TraceQL query set.
- Service graph compared with architecture.
- Runbook and measured investigation time.

Interview Questions and Model Answers

Q: What is the value of exemplars?

A: They connect an aggregated metric point to a concrete trace that contributed to the observation.

Q: Can span metrics replace application metrics?

A: Sometimes they supplement them, but sampling and instrumentation quality can bias the generated metrics.

Chapter Summary

- TraceQL enables structural and attribute-based trace investigation.
- Cross-signal links turn separate tools into one diagnostic workflow.
- Correlation quality depends on consistent identity and context.

VOLUME 11D | CHAPTER 38

Chapter 38: Application Performance Monitoring and Production Diagnosis

Combining service health, runtime behavior, dependencies and request evidence

APM Operating Model

Application Performance Monitoring is not a single agent or dashboard. It is a workflow that combines service-level indicators, request traces, runtime metrics, dependency telemetry, profiles, logs, releases and ownership data. The platform should support detection, triage, localization, explanation, mitigation and verification.

Layer	Primary Questions	Evidence
User experience	Is the service meeting user expectations?	Availability, latency, correctness and synthetic checks.
Service	Which route, version, tenant class or region is affected?	RED metrics, traces and deployment metadata.
Dependency	Which database, cache, queue or external API contributes?	Client spans, saturation metrics and timeout logs.
Runtime	Is CPU, memory, GC, thread or event-loop behavior abnormal?	Runtime metrics and continuous profiles.
Infrastructure	Is node, container, network or policy responsible?	Host metrics, Kubernetes events, Hubble flows and eBPF evidence.
Code	Which function or line consumes resources?	Flame graphs, allocation profiles and span profiles.

Structured Diagnostic Method

1. Confirm user impact and define the affected time, service, environment and release.
2. Use SLO or RED metrics to identify whether errors, latency or saturation dominate.
3. Open representative traces from exemplars or trace search.
4. Identify the longest critical-path spans and compare successful versus failed requests.
5. Correlate dependency metrics, logs and network flows for the same interval.
6. Use profiles when CPU, allocation, lock or runtime behavior remains unexplained.
7. Apply the smallest reversible mitigation and verify recovery using the original user-facing signal.

Anti-Patterns

- Starting with host CPU without confirming user impact.
- Treating the longest span as the root cause without checking parallel work and critical path.
- Using averages that hide tail latency.
- Adding unbounded attributes during an incident and causing a telemetry cost incident.
- Assuming an APM agent is harmless without measuring overhead and failure isolation.
- Closing the incident when dashboards turn green without validating user transactions and backlog recovery.

Evidence-driven hypothesis template

Incident hypothesis table

Hypothesis: database connection pool exhaustion
 Evidence for: DB client spans wait before query; pool saturation metric high
 Evidence against: database CPU and query time normal
 Test: increase pool only in canary or reduce concurrency
 Success signal: p99 latency and queue wait return to baseline
 Rollback: restore previous concurrency and pool configuration

Production Readiness Checklist

- Define standard APM dashboards and investigation paths.

- Correlate release version across all signals.
- Monitor instrumentation overhead and agent health.
- Create ownership and runbook links per service.
- Practice evidence-based diagnosis with controlled failures.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
APM shows latency but no cause	Missing dependency spans or profile data	Inspect trace critical path and coverage	Add targeted instrumentation or profiling.
Agent causes instability	Unsupported runtime or excessive telemetry	Compare canary resource and latency	Rollback agent and tune or upgrade.
Trace and metric disagree	Different dimensions, sampling or clocks	Compare definitions and time windows	Document source-of-truth signal and fix identity/time alignment.
Mitigation hides symptom only	Backlog or dependency remains unhealthy	Check end-to-end transaction and queues	Continue recovery until user path and deferred work stabilize.

Hands-On Lab

1. Create three failures: slow DB, CPU hotspot and denied network request.
2. Diagnose each using the structured workflow.
3. Record time to detection and time to localization.
4. Apply reversible mitigation and verify user recovery.
5. Write one incident note with evidence for and against each hypothesis.

Validation Evidence

- Three incident timelines.
- Trace, metric, log, profile or flow evidence for each root cause.
- Mitigation and rollback commands.
- Post-incident observability improvement list.

Interview Questions and Model Answers

Q: What is the difference between monitoring and APM?

A: APM connects user and service performance to request, dependency, runtime and code-level evidence for diagnosis.

Q: Why compare failed and successful traces?

A: The difference often reveals the abnormal dependency, path, version or timing while controlling for normal structure.

Chapter Summary

- APM is an investigation workflow, not a product label.
- Use user impact to guide the order of evidence.
- Mitigation must be verified against the original user-facing signal.

VOLUME 11D | CHAPTER 39

Chapter 39: Continuous Profiling with Grafana Pyroscope

Finding expensive code paths and intermittent resource problems in production-like workloads

Why Continuous Profiling

Continuous profiling samples application behavior over time and aggregates stack traces into profiles. Unlike one-off debugging, it captures intermittent and workload-dependent hotspots. Common profile types include CPU, wall time, allocations, heap, goroutines or threads, mutex or lock contention, and language-specific runtime profiles. Availability varies by runtime and collection method.

Profile Type	Question	Common Interpretation Risk
CPU	Which code consumes CPU cycles?	High cumulative cost may be expected hot-path work.
Wall time	Where is elapsed execution time spent?	Includes blocking and waiting, not only CPU.
Allocations	Which code allocates most memory over time?	High allocation does not always mean retained memory.
Heap / in-use	Which objects remain live?	GC timing and runtime behavior affect snapshots.
Lock / mutex	Where does contention occur?	Sampling may miss rare short contention.
Goroutine / thread	Which execution paths create or hold workers?	Count alone does not prove a leak.

Pyroscope and Label Design

Pyroscope stores and queries profiles and integrates with Grafana. Labels should allow comparison by service, environment, version, cluster and controlled workload dimension. Avoid request IDs, pod UUIDs, user IDs or raw paths as labels. High-cardinality profile labels create cost and query problems similar to metrics and Loki labels.

Profile comparison contract

```
# Conceptual profile identity
service_name="checkout-api"
environment="production"
service_version="2026.08.1"
cluster="ap-south-primary"
profile_type="process_cpu"

# Comparison question
Compare checkout-api CPU profile:
  version=2026.08.1
against:
  version=2026.07.4
```

Flame Graph Analysis

- Width represents the amount of sampled resource attributed to a stack path, not elapsed order.
- The top of a stack is the current leaf function; the bottom is the entry path.
- Compare profiles between good and bad windows, versions, regions or feature flags rather than reading one graph in isolation.
- Validate symbols and stack unwinding. Unknown frames can make an apparent hotspot misleading.
- Use profiles to form a code hypothesis, then reproduce or benchmark before making a risky optimization.

Tracing and Profiling

Trace-to-profile integrations narrow profile analysis to the service and time interval of a span. Span profiles can provide request-focused analysis in supported runtimes. Use this to distinguish a globally hot function from the code path active during one slow request. Keep service identity and version labels consistent across Tempo and Pyroscope.

Production Readiness Checklist

- Select profile types by runtime and diagnostic need.
- Control profile labels and retention.
- Measure profiler CPU and memory overhead.
- Validate symbolization and stack quality.
- Integrate profiles with service, version and trace context.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Flame graph shows unknown frames	Missing symbols or unwinding limitations	Compare binary build flags and profiler logs	Publish symbols, enable supported unwinding or use language profiler.

Symptom	Likely Cause	Verification	Corrective Action
No profiles for service	Wrong endpoint, labels or unsupported runtime	Inspect agent targets and ingestion metrics	Correct discovery, credentials or collector method.
Profiler overhead high	Sampling frequency or profile types excessive	Compare canary resource use	Reduce frequency/types or switch method.
Version comparison impossible	service_version absent or inconsistent	Inspect profile labels	Standardize resource and profile identity.

Hands-On Lab

1. Deploy Pyroscope and instrument one CPU-intensive service.
2. Generate a controlled hotspot and capture baseline and bad-window profiles.
3. Compare two versions or feature modes.
4. Correlate one slow trace interval with a profile.
5. Fix the hotspot and verify CPU and latency improvement.

Validation Evidence

- Baseline and regression flame graphs.
- Profile labels and retention configuration.
- Overhead measurement.
- Before-and-after CPU and latency results.

Interview Questions and Model Answers

Q: What does flame graph width represent?

A: The proportion of sampled resource attributed to that stack path, not chronological execution order.

Q: Why use differential profiles?

A: Comparing good and bad conditions highlights what changed and reduces noise from normal hot paths.

Chapter Summary

- Continuous profiling captures intermittent code-level resource behavior.
- Label governance is as important for profiles as for metrics.
- Profiles generate hypotheses that should be validated with workload evidence.

VOLUME 11D | CHAPTER 40

Chapter 40: eBPF Observability and Full-Stack Telemetry Capstone

Using kernel-level evidence responsibly and integrating every observability signal

eBPF Observability Scope

eBPF runs sandboxed programs in the Linux kernel and supports networking, security and performance observability without modifying kernel source. Observability products use eBPF to collect network flows, protocol details, system calls, CPU profiles or automatically inferred service activity. It is powerful but not magic: coverage depends on kernel, runtime, privileges, symbols, protocol encryption and tool design.

Use Case	Example Evidence	Limitation
Network flows	Source, destination, identity, verdict, DNS and protocol metadata	Encrypted payload and unsupported protocol detail may be unavailable.
Policy diagnosis	Which network policy allowed or denied a flow	Requires compatible CNI and policy correlation.

Use Case	Example Evidence	Limitation
CPU profiling	System-wide sampled stacks	Linux only; profile types and language support vary.
Auto-instrumentation	Inferred HTTP or service activity without source change	Business semantics and deep framework context may be limited.
Security observability	Process, file, network or syscall events	High event volume, privilege and policy tuning requirements.

Hubble Network Visibility

Hubble is the observability layer of Cilium and provides cluster-wide network and security visibility. It can show service maps, flow verdicts, DNS-aware data and Prometheus metrics. Hubble Relay aggregates node-level visibility. Protect Relay and UI access, define retention expectations, and validate that network identity maps correctly to Kubernetes workloads.

Hubble laboratory commands

```
# Enable Hubble in a lab cluster
cilium hubble enable

# Inspect recent denied flows
hubble observe --verdict DROPPED --since 5m

# Filter by namespace and destination port
hubble observe --namespace commerce --to-port 5432

# Validate platform health
cilium status
hubble status
```

eBPF Profiling with Grafana Alloy

Grafana Alloy can collect eBPF CPU profiles and send them to Pyroscope. The method requires Linux and commonly elevated host access. It does not provide all profile types, and unsupported languages or missing symbols reduce usefulness. Treat host capabilities, privileged mode, kernel interfaces and mounted paths as security-sensitive deployment decisions.

Privilege Gate Do not deploy eBPF collectors merely because they are convenient. Require security review, documented host capabilities, image provenance, kernel compatibility, network egress restriction, upgrade rollback and a clear statement of collected data.

Capstone Architecture

Plane	Components	Acceptance Evidence
Instrumentation	Manual and auto OpenTelemetry instrumentation	Complete trace across HTTP and queue boundaries.
Collection	Agent and gateway Collectors	Queue, retry, privacy processing and internal telemetry validated.
Tracing	Tempo, queue, object storage and Grafana	Trace search, retention, auth and outage recovery tested.
Metrics	Prometheus and span metrics	RED dashboard, SLOs and exemplars.
Logs	Loki or Elastic/OpenSearch	trace_id correlation and privacy validation.
Profiles	Pyroscope and application/eBPF profiles	Differential profile identifies controlled hotspot.
Network	Cilium Hubble	Denied flow and DNS or service path diagnosis.

Plane	Components	Acceptance Evidence
Operations	Alerts, runbooks, ownership and incident record	Timed diagnosis, mitigation, rollback and post-incident actions.

Capstone Failure Scenarios

1. Propagation break between producer and consumer.
2. Slow database dependency affecting only one application version.
3. Collector gateway backend outage causing queue growth.
4. Tempo downstream consumer lag or object-storage failure.
5. CPU hotspot introduced by a feature flag.
6. Kubernetes network policy denying an application dependency.
7. Unsafe attribute or baggage value blocked before export.
8. Telemetry overload requiring emergency sampling and restoration.

Production Readiness Checklist

- Approve eBPF privilege and data collection scope.
- Secure Hubble Relay, UI and profile exporters.
- Integrate consistent service, environment and version identity across all signals.
- Run every capstone failure and recovery drill.
- Document SLOs, capacity, backup, rollback and ownership for the observability platform.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Hubble shows no flows	Hubble disabled, Relay unavailable or filter too narrow	Run cilium and hubble status	Enable components, repair certificates or broaden filter.
Denied flow has unclear owner	Identity or workload labels incomplete	Inspect flow labels and Kubernetes objects	Fix workload metadata and ownership mapping.
eBPF profiler has unknown stacks	Unsupported runtime or symbolization problem	Review profiler and binary information	Use supported runtime settings or language-native profiler.
Signals cannot correlate	Different service/version/environment naming	Compare labels and resources across backends	Apply one identity contract at instrumentation and collection.

Hands-On Lab

1. Build the integrated lab platform described above.
2. Deploy a three-service application with HTTP, queue and database dependencies.
3. Execute all eight failure scenarios and measure diagnosis time.
4. Create a production-readiness review with security, capacity and recovery evidence.
5. Present one incident from symptom through mitigation, verification and prevention.

Validation Evidence

- Architecture diagram and version inventory.
- Sanitized configurations for instrumentation, Collectors and backends.
- Dashboards and cross-signal navigation evidence.
- Eight failure drill records with timestamps.
- Final acceptance checklist and improvement backlog.

Interview Questions and Model Answers

Q: What is the main advantage of eBPF observability?

A: It can provide kernel-level networking and performance evidence with little or no application code change.

Q: What is the main risk?

A: It often needs elevated host access and has portability, coverage, privacy and symbolization limitations.

Q: What proves the capstone is production-ready?

A: Repeatable deployment, secure access, capacity evidence, cross-signal diagnosis, tested failure recovery, rollback and documented ownership.

Chapter Summary

- eBPF complements rather than replaces application instrumentation.
- The capstone must prove diagnosis and recovery, not only data ingestion.
- A shared identity contract is the foundation of full-stack correlation.

Volume 11D Completion Checklist

- I can explain trace IDs, span IDs, parent-child relationships, links, events, span kinds, resources and propagation.
- I can diagnose broken context across HTTP, gRPC, queues and background jobs.
- I can define a resource and semantic-convention contract across teams.
- I can select manual, library, zero-code, Operator or eBPF instrumentation and explain the trade-offs.
- I can configure and validate an OpenTelemetry Collector pipeline with memory limiting, privacy processing, batching, queueing, retry and health checks.
- I can model Collector and tail-sampler capacity and explain stateful scaling.
- I can design safe head and tail sampling policies with a healthy baseline.
- I can operate Tempo with authenticated access, tenant controls, object storage, queue monitoring, retention and recovery.
- I can use TraceQL, exemplars, service graphs and span metrics in an incident workflow.
- I can analyze differential flame graphs and correlate profiles with traces and versions.
- I can use Hubble to diagnose network flows and policy drops and explain eBPF privilege limitations.
- I have completed the integrated capstone and retained sanitized evidence of every failure drill.

Ten Portfolio Projects

1. Trace continuity validator for HTTP and messaging boundaries.
2. Organization-wide OpenTelemetry resource and semantic convention contract.
3. Kubernetes Operator auto-instrumentation canary with overhead report.
4. Agent-to-gateway Collector platform with persistent queue outage drill.
5. Tail-sampling tier preserving errors, slow traces and a random baseline.
6. Tempo production design with Kafka-compatible queue, object storage and tenant limits.
7. Grafana trace-to-logs, trace-to-metrics and exemplar navigation package.
8. APM incident lab covering DB delay, runtime saturation and release regression.
9. Pyroscope differential profiling project with a verified code optimization.
10. Cilium Hubble network-policy incident plus full-stack observability capstone.

30-Day Practice Plan

Days	Focus	Deliverable
1-3	Trace model, propagation and asynchronous links	Three-service trace and broken-context drill.
4-6	Resources, semantic conventions and schema governance	Telemetry contract and consistency query.
7-9	Manual and zero-code instrumentation	Canary rollout and overhead report.
10-13	Collector agent, gateway, queue, retry and internal telemetry	Outage drill and capacity model.
14-16	Head and tail sampling, baggage and privacy	Sampling comparison and allowlist test.
17-20	Tempo deployment, security, storage and operations	Backend design and recovery exercise.
21-23	TraceQL, exemplars and service graphs	Incident query pack and correlation runbook.
24-26	APM diagnosis and runtime evidence	Three controlled incident reports.
27-28	Pyroscope profiling	Differential flame graph and optimization proof.
29-30	eBPF, Hubble and capstone	Network-policy drill and final platform review.

Command and Configuration Reference

Task	Example
Validate Collector config	<code>otelcol validate --config=/etc/otelcol/config.yaml</code>
Check Collector health	<code>curl -fsS http://localhost:13133/</code>
OTLP gRPC default port	4317
OTLP HTTP default port	4318
Query Tempo API with CLI	<code>tempo-cli query api search <host:port> <TraceQL></code>
Check Cilium	<code>cilium status</code>
Enable Hubble lab	<code>cilium hubble enable</code>
Inspect dropped flows	<code>hubble observe --verdict DROPPED --since 5m</code>
Search logs by trace ID	Use <code>trace_id</code> field in Loki or search backend
Compare profiles	Select same service and time windows with different version labels

Glossary

Term	Meaning
APM	Operational practice combining service, request, dependency, runtime and code-level performance evidence.
Baggage	Key-value contextual data propagated alongside trace context; it requires strict privacy control.
Exemplar	Reference from an aggregated metric observation to a representative trace.
OTLP	OpenTelemetry Protocol for transporting traces, metrics, logs and supported telemetry.
Resource	Attributes identifying the entity that produced telemetry.
Semantic conventions	Standard attribute names and operation meanings defined by OpenTelemetry.
Span	Timed unit of work inside a trace.
Tail sampling	Sampling decision made after trace data has been collected.
TraceQL	Tempo query language for selecting and analyzing traces.
Continuous profiling	Ongoing collection and analysis of sampled stack and runtime resource behavior.
eBPF	Linux kernel technology used for programmable networking, security and performance observation.
Hubble	Cilium observability layer for cluster network and security flows.

Official Documentation Index

- [OpenTelemetry Concepts](#)
- [OpenTelemetry Semantic Conventions](#)
- [OpenTelemetry Zero-Code Instrumentation](#)
- [OpenTelemetry Collector Configuration](#)

- [OpenTelemetry Collector Deployment](#)
- [OpenTelemetry Collector Scaling](#)
- [OpenTelemetry Collector Internal Telemetry](#)
- [OpenTelemetry Sampling](#)
- [OpenTelemetry Security](#)
- [Grafana Tempo Documentation](#)
- [Tempo Architecture](#)
- [Tempo Deployment](#)
- [TraceQL](#)
- [Tempo Metrics from Traces](#)
- [Grafana Pyroscope](#)
- [Continuous Profiling](#)
- [Pyroscope eBPF with Alloy](#)
- [Cilium Hubble](#)
- [Hubble Setup](#)
- [eBPF Project](#)

Version Verification Product versions, configuration schemas, feature maturity, license terms and support status can change. Confirm every production command and option against the official documentation for the exact pinned version.

Volume 11 Status

Part	Status
11A	Completed
11B	Completed
11C	Completed
11D	Completed with this book
11E	Next: SRE Operations, Incidents, Capacity and Reliability Governance
11F	Planned: Enterprise Projects, Certifications, Interviews and Final Capstone

Next Step Proceed to Volume 11E to turn observability capabilities into disciplined on-call operations, incident command, reliability engineering, capacity planning, chaos practice, governance and continuous improvement.